Adaptive Compute at Matched Parameters: Routing and Recursion Trade Off in Small Language Models

Anonymous ACL submission

Abstract

Conditional computation chooses either *which* parameters process a token —sparse mixture-of-experts (MoE)—or *how much* computation it receives —adaptive-depth weight-tied recursion (MoR). Composing them should inherit both. We test whether it pays: one engine, three configurations, parameters matched to 0.069% (≈5.58M), bit-identical inputs, five seeds, exact randomisation tests under a pre-registered correction, on WikiText-103.

All three arms lie on the quality/compute frontier, and the composition (MoRE) buys the most quality per unit of compute: 7.9% better validation loss than MoE at 2.19× its inference FLOPs where recursion alone needs 9.07×—72% of recursion's gain for 24% of its compute. Out of domain it *dominates* recursion alone, reaching lower loss at a quarter of the compute. It is not lowest-loss in domain: recursion alone leads by 0.1216 nats, significant at our resolution floor, and routing improves neither in-domain quality nor routing quality over MoE, so we keep the quality and compute axes separate. We trace the deficit to a competition for one per-token budget, excluding two rival explanations. Most transferably: a mean step count and an exit histogram cannot establish input-dependent depth, since a uniform policy forced to the learned mean reproduces both—and scoring against that control separates our recursive arms by a factor of 24.

1 Introduction

Conditional computation is motivated by one accounting argument—most tokens do not need the whole model—made in two places that answer different questions. Sparse mixture-of-experts (MoE) layers (Jacobs et al., 1991; Shazeer et al., 2017; Fedus et al., 2022) decide *which* computation module processes a token; recursive weight-tied architectures with learned halting (Graves, 2016; Dehghani et al., 2019; Banino et al., 2021) decide *how much* computation it receives, reusing one block for a variable number of steps. The decisions are over different objects, so they are orthogonal in principle and composing them is an obvious thing to try: route a token to a specialised expert, then reuse *that expert's* weights for as many steps as it appears to need. The composition inherits an efficiency argument from each side. What it does not inherit is evidence—whether it beats either component at a matched budget has not, to our knowledge, been isolated.

We isolate it. One training engine implements three configurations differing only in the fields their definitions require: routing without recursion (MoE, six experts, depth 1), recursion without routing (MoR, one shared block, depth up to 7), and the composition (MoRE, six experts, depth up to 7). All three sit at ≈5.58M parameters, share a bit-identical input representation, and train on the same fixed split of WikiText-103 (Merity et al., 2017) with the same optimiser budget, so a difference between arms cannot be a difference in the harness, the data, or the parameter count.

What we find. The composition occupies the compute-efficient point of the three, and the honest accounting of what it does and does not buy is the paper.

- **All three arms sit on the quality/compute frontier, and the composition buys the most quality per unit of compute.** MoRE improves on MoE by 7.9% validation loss (3.948 → 3.637 nats/token) at 2.19× MoE's inference FLOPs where MoR needs 9.07×: 72% of MoR's gain for 24% of its compute, roughly 3× the quality per unit compute. Out of domain (Gao et al., 2021) the ratio becomes outright *dominance*—MoRE reaches 7.573 nats against MoR's 7.802 at a quarter of the compute, so MoR is Pareto-dominated—which is the one respect in which composing the mechanisms

does something neither does alone.

- **But recursion alone remains the lowest-loss arm in domain.** MoR reaches 3.515 against MoRE's 3.637: the composition trades 0.1216 nats, significant at our resolution floor, for a $4\times$ inference-FLOP saving. These are two axes and we never substitute one for the other. Adding routing improves neither in-domain quality over MoR nor routing quality over MoE.
- **Recursion's adaptive depth is real; the composition's barely survives.** Against a *depth-matched* uniform policy—forced to the learned policy's own mean depth, so total compute is fixed—MoR's halting buys 1.47 nats and MoRE's 0.06, a factor of 24. MoRE's halting has collapsed to running to budget on 97.3% of tokens.
- **A diagnostic that generalises.** A mean step count and an exit histogram—what this literature almost always reports—do *not* establish input-dependent depth, since a policy ignoring its input reproduces both. Scoring against a depth-matched uniform policy does, costs one inference pass, and is what separates our two arms, which are otherwise indistinguishable.
- **The two mechanisms compete for one budget, and two rival explanations are excluded.** A sweep that recovers adaptive depth by raising the ponder penalty destroys expert differentiation, while the arm that preserves differentiation saturates depth. Capacity dilution is ruled out arithmetically—tokens per parameter are identical by construction—and subspace trapping is falsified three ways (§6).

We report this as a trade-off measured at this scale, not a verdict on either mechanism in general. The contribution is the decomposition: because MoE *is* MoRE with the depth budget set to one and all three arms are parameter-matched from one codebase, each contrast varies one thing.

2 Related Work

Conditional mixtures of experts date to Jacobs et al. (1991); the sparsely-gated form is due to Shazeer et al. (2017), with the Top-1 dispatch and load-balancing machinery of GShard (Lepikhin et al., 2021), Switch Transformer (Fedus et al., 2022) and ST-MoE (Zoph et al., 2022)—whose standard justification, more parameters at constant FLOPs, is unavailable here since we pin all three arms to one parameter count. On the depth axis, Adaptive Computation Time (Graves, 2016) supplies the halting unit we use, Universal Transformers (Dehghani et al., 2019) applied it to a weight-tied block, PonderNet (Banino et al., 2021) reformulated the halting distribution to keep a gradient near saturation, and early-exit variants place the decision at the layer or token level (Elbayad et al., 2020; Schuster et al., 2022; Raposo et al., 2024). That literature reports mean step counts and exit histograms almost universally, which §5.2 argues cannot test input-dependence. Closest to us, Bae et al. (2025) and Raposo et al. (2024) route *to depths* over shared weights; MoRE instead routes among independent expert FFNs whose weights the recursion then reuses, and we know of no matched-budget comparison of such a composition against each of its components—the gap we fill. We claim no novelty for the combination itself. Minimal-pair and likelihood probes (Warstadt et al., 2020; Ivanova et al., 2024) score a *relative* preference within a matched pair, so they degrade gracefully with scale, which is what makes them usable at 5.6M parameters.

3 The MoRE Architecture

3.1 Setting and notation

The task is autoregressive language modelling on packed token sequences. Write E for the number of experts, T for the maximum recursion depth, d for the model width, and $\mathbf{h}_s^{(0)} \in \mathbb{R}^d$ for the post-embedding state of position s. Unlike a synthetic benchmark, natural language supplies *no per-token oracle*—neither an expert index nor a complexity label—so every structural claim below must rest on permutation-invariant statistics or on comparisons against a controlled alternative. This is why §5.2 is built around a forced-depth sweep rather than around agreement with ground truth.

3.2 Routing

Routing happens once per token. A linear router produces $\mathbf{g} = \mathrm{softmax}(W_r \mathbf{h}_s^{(0)}) \in \Delta^{E-1}$, the token is assigned to $e^\star = \arg\max_e g_e$, and *only* that expert is evaluated. The assignment is *persisted*: a token keeps one expert for its whole recursion trajectory, so the block below reuses one expert's parameters at every depth and one halt head observes a coherent sequence of states. (An earlier configuration re-drew both per step; it is retained as a labelled ablation and discussed in the disclosures

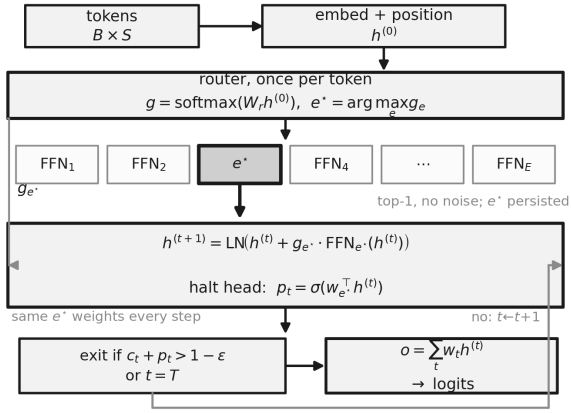


Figure 1: The MoRE block. A token is routed *once* from its post-embedding state to one of E independent expert FFNs, and *that expert's* weights—never another's—are reused for an adaptive number of recursion steps under ACT halting. Only the selected expert is evaluated, and its output is scaled by the gate $g_{e^\star}$. Setting $T = 1$ recovers MoE exactly and setting $E = 1$ recovers MoR; the input representation is bit-identical in all three cases, and all three are configurations of one engine.

in the Limitations section.)

$$\mathbf{h}^{(t+1)} = \mathrm{LayerNorm}\big(\mathbf{h}^{(t)} + g_{e^\star}\,\mathrm{FFN}_{e^\star}(\mathbf{h}^{(t)})\big) \tag{1}$$

for $t = 0, \ldots, T-1$.

The gate probability $g_{e^\star}$ multiplies the expert output, which is what carries task-loss gradient into the router: the discrete $\arg\max$ is not differentiated, but the scalar scaling the selected expert's contribution is. There is one block, so depth is computation through time rather than a stack of depth-specific layers, and no capacity limit is imposed—every routed token is evaluated by its expert and none is dropped.

3.3 Halting and the training objective

Halting follows ACT (Graves, 2016) as used in the Universal Transformer, with one halt head per expert. Per token, at step t:

$$p_t = \sigma\big(\mathbf{w}_{e^\star}^\top \mathbf{h}^{(t)}\big), \quad c_t = \textstyle\sum_{\tau \le t} p_\tau, \tag{2}$$

$$\text{halt when } c_t + p_t > 1 - \epsilon, \tag{3}$$

$$R = 1 - c_t, \quad w_t = \begin{cases} p_t & \text{running} \\ R & \text{at halt} \end{cases} \tag{4}$$

with $\epsilon = 0.01$ and R the halting remainder.

and the token's output is the convex combination $\mathbf{o}_s = \sum_t w_t \mathbf{h}^{(t)}$, whose weights sum to exactly 1. That sum-to-one property is the mechanism by which halting is trained: because w_t scales how much of each step's state reaches the output, changing p_t changes the task loss, so a differentiable path runs from $\mathcal{L}_{\text{task}}$ into the halt parameters. The boolean comparison in Eq. 3 decides dispatch only. Tokens still active at $t = T$ are forced out; halted states are frozen and not recomputed, so a halted token costs nothing at deeper steps.

Depth is trained through the expected-depth term

$$\rho \;=\; \mathbb{E}_{\text{tokens}}\big[\,N + R\,\big], \tag{5}$$

where the step count N is discrete and contributes no gradient while the remainder R does. Eq. 5 is the ponder cost. It is worth noting that the gradient reaching the halt heads through R is small compared with the task gradient: a direct measurement in our implementation puts the halt gradient at roughly $285\times$ below the task term. This ratio is the quantitative form of the failure analysed in §6: the ponder term is large enough to be optimised away but too weak to hold depth down once the task gradient accumulates, and once the halting sigmoid saturates the gradient through R vanishes (Graves, 2016). Banino et al. (2021) is the principled fix and is future work (the Limitations section).

The training objective is a weighted sum of terms that we report separately and never as a single number:

$$\begin{aligned}\mathcal{L} \;&= 1.0\,\mathcal{L}_{\text{task}} + 0.001\,\mathcal{L}_{\text{bal}} + 0.001\,\rho \\ &\quad + 0.0\,\mathcal{L}_{\text{fam}} + 0.0\,\mathcal{L}_{\text{route}}.\end{aligned} \tag{6}$$

$\mathcal{L}_{\text{task}}$ is per-token cross-entropy in nats. $\mathcal{L}_{\text{bal}} = -H(\bar{\mathbf{g}}) + E\sum_e f_e \bar{g}_e$ combines a router-entropy bonus with the Switch-style auxiliary (Fedus et al., 2022), where f_e is the dispatched load fraction and $\bar{\mathbf{g}}$ the mean soft router probability; it is averaged over the depth calls actually made, so its magnitude does not grow with T. Three terms are structurally zero on language and the reason is not convenience. $\mathcal{L}_{\text{fam}}$ reads $\mathbf{h}.\mathrm{detach}()$, so its gradient cannot reach the trunk—which *strengthens* every specialisation claim below relative to a comparable synthetic study, where a family cross-entropy is typically one of the larger terms in the objective. $\mathcal{L}_{\text{route}}$ and the halting-supervision term are zero because natural language supplies no per-token oracle expert and no per-token oracle depth to supervise against. The canonical router is therefore unsupervised with respect to the per-token routing decision.

3.4 One engine, three modes

MoE, MoR and MoRE are configurations of one implementation with an explicit `architecture` mode, so a difference between arms cannot be a difference in the harness. Equalising *total* FFN hidden width gives MoR's single FFN a multiplier of $24 = 4 \times 6$: **5,584,908 parameters for MoE and MoRE, identical, and 5,581,063 for MoR**—a 0.069% deficit that is the router MoR lacks, and which runs *against* our headline since the lighter arm wins in domain. Appendix F derives it.

4 Experimental Setup

4.1 Data and split

All arms train on WikiText-103 (Merity et al., 2017) with a byte-pair vocabulary of 8,192 tokens and packed sequences of length 256 (identifier in Appendix I). We use the corpus's own author-provided validation split rather than carving one out of the training material, which is what makes the leakage audit in §4.5 meaningful: no validation or test token was ever available to the trainer.

The baseline floor is a backoff bigram estimated on the same corpus: **4.9849 nats/token** (7.19 bits, perplexity 146.2). A model that does not beat it has learned nothing a two-column count table could not. All three arms beat it, by margins of $+1.037$, $+1.470$ and $+1.348$ nats for MoE, MoR and MoRE respectively.

4.2 Training protocol

Five seeds per arm; 3 epochs of AdamW (lr 10^{-3}, weight decay 10^{-4}, clip 1.0, dropout 0.1), $d_{\text{model}} = 256$, one block, 4 heads, $T = 7$, batch 48, on a single laptop GPU. Settings were fixed once, before the comparison, and not swept per architecture. **Three epochs is a fixed compute budget, not convergence**—every arm gets the identical budget, so the comparison is valid though no arm is converged (Appendix I).

4.3 Metrics

The primary metric is **validation task loss**, final-epoch per-token cross-entropy in nats; `best` equals `last` on all 15 cells, so *no model selection occurred*. Routing is scored with permutation-invariant measures (AMI, Hungarian, purity) against a per-run shuffled-label control and reported as a z-score. Depth is reported with its exit rates alongside: a depth–difficulty correlation on an arm whose tokens almost all reach the ceiling is taken against a near-constant and **says nothing about allocation**. Appendix G defines each.

4.4 Statistics

Comparisons use an exact two-sided randomization test on the difference of means, which enumerates every way of splitting the pooled values into groups of the original sizes—$\binom{10}{5} = 252$ at five-versus-five—and counts the fraction whose |mean difference| is at least the observed one. It is exhaustive, hence deterministic: no sampling error, no seed, no normality assumption, and no threshold to choose. The superseded $k\times$std heuristic is not used anywhere.

The **resolution floor** of this design — the smallest p it can return at all — is $2/\binom{n_a+n_b}{n_a}$ for equal arms, which is $2/252 = 0.00794$ at five-versus-five. The factor of two is there because the two-sided statistic is invariant under swapping the groups, so extreme regroupings arrive in mirror pairs and the count at the extreme can never be one. The derivation is in Appendix A. **Every headline comparison in this paper sits exactly on that floor**, which we phrase as "as extreme as a five-seed design admits"—a weaker statement than significance, and not to be dressed up as a stronger one. Only more seeds can strengthen these claims; no larger effect can.

The correction is Holm–Bonferroni over two pre-registered families (predictive quality, $m = 4$; adaptive depth, $m = 2$), declared before the final arm was trained. Appendix B gives the family memberships and explains why plain Bonferroni would be unusable here.

4.5 Integrity checks

Three invariants hold across the matrix, each enforced by an automated check: inputs are **bit-identical across arms** for the same record; the target is absent from the input and no oracle routing label is encoded in a feature slot; and the arms are parameter-matched to 0.069% with seed-blind configurations identical within each arm, so comparability rests on a frozen specification rather than on one code revision. Appendix C lists them with the structurally-zero loss terms.

5 Results

5.1 Predictive quality

Table 1 collects every result. Recursion alone is lowest-loss. MoR beats MoE by 0.4329 nats and

beats MoRE by 0.1216. Both recursive arms beat MoE decisively, and the composition lands between them—the premise that routing would *add* to recursion in domain is not supported.

Two features of the table deserve to be read together. First, MoE and MoRE are parameter-*identical* at 5,584,908 while MoR sits 0.069% lower, the residual being the router MoR does not have—and that residual runs *against* the headline, since the arm with fewer parameters is the one that wins. Second, MoRE's seed spread is $6.4\times$ tighter than MoR's in variance (std 0.0134 against 0.0340; exact one-sided permutation test on the variance ratio $p = 1/252$, that test's floor). **What the number means is still open**: these runs cannot separate a genuinely stabilising effect from the simpler account that a model whose halting has collapsed to a near-constant has less left to vary, and the second reading is the more parsimonious. We report the measurement and decline the interpretation. (Exploratory and uncorrected.)

Fig. 2 and the forced-depth sweep sharpen this. Both arms scale with forced depth up to a point, in opposite directions:

- **MoR** improves from 7.36 nats at depth 1 to a minimum of 4.06 at depth 4, then *degrades* to 4.99 at depth 7. It has a genuine interior optimum.
- **MoRE** improves monotonically all the way to depth 7 (3.70), never turning over within the budget. It wants more depth than it has.

5.2 Is recursion actually adaptive?

The mechanism claim rests on comparing each learned policy against a *uniform* policy at the same compute, which is the cheapest competitor that ignores its input and therefore the minimum bar an adaptive-depth claim must clear. Two comparisons matter, and they give different answers for the two arms.

Against the best uniform depth anywhere (the strongest competitor): MoR's learned policy at 3.5154 beats its best forced depth (4.0558 at d=4) by **0.540 nats**. MoRE's learned policy at 3.6370 beats its best forced depth (3.6971 at d=7) by **0.060 nats**.

Against a depth-matched uniform policy—forced to exactly the arm's own measured mean depth of 6.955, so the comparison isolates *where* the compute goes rather than *how much* there is: MoR's adaptive policy beats forced-d7 (4.9863) by **1.471 nats**; MoRE's beats forced-d7 (3.6971) by **0.060 nats**.

The depth-matched comparison is the one that licenses a mechanism claim, because best-depth comparison conflates adaptivity with the average compute spent. On it, both arms' halting is doing something real and measurable—but MoR's buys 1.47 nats where MoRE's buys 0.06, a factor of roughly 24. The honest summary is not that MoRE's depth mechanism is inert; it is that adding routing left it almost entirely without effect.

The depth-matched control, as a general diagnostic. A mean step count and an exit histogram—the two statistics this literature almost always reports—do not establish that depth depends on the input: a policy ignoring its input reproduces any mean and, with a little variance, a plausible histogram. The comparison that does is against a *uniform policy forced to the learned policy's own mean depth*, which holds total compute fixed so the only thing left to differ is *where* it was spent. It costs one inference pass, needs no retraining, and separates our two arms—both of which look adaptive by mean-and-histogram, and only one of which is. We recommend it before any model is described as allocating computation adaptively.

The exit statistics say why MoRE's margin is so small. MoR early-exits $18.8\% \pm 15.1\%$ of validation tokens with mean depth 6.730 and a halt remainder of 0.175; MoRE force-exits $97.3\% \pm 0.8\%$ at the ceiling with mean depth 6.955 and remainder 0.655. MoRE's halting has collapsed to "run to budget".

The depth–difficulty correlation is reported next to those rates because it is only interpretable alongside them. MoR's per-token depth correlates with per-token model loss at $\rho = +0.2985 \pm 0.0654$, positive on **5/5** seeds. MoRE's is -0.0230 ± 0.0115—but with 97.3% of tokens exiting at the ceiling and a per-token depth standard deviation of 0.43, that correlation is computed against a near-constant. We report it as *uninformative*, not as evidence that MoRE allocates depth badly. This distinction is the paper's most load-bearing methodological point: a near-zero correlation with a constant is not a finding about allocation.

MoR's early-exit rate is the seed-unstable quantity (6.4%–42.2%), and its instability is informative: the two seeds that exit most are the two

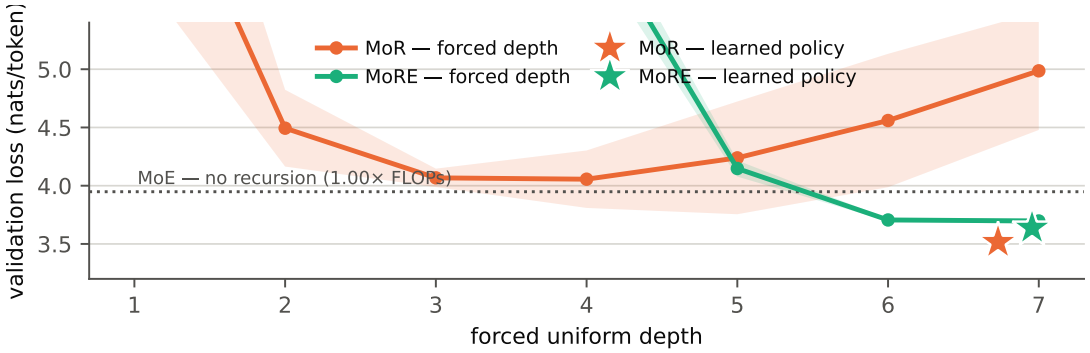


Figure 2: Validation loss against forced uniform recursion depth, five seeds. Lines are the forced-depth sweep; stars mark each arm's *learned* policy at its own measured mean depth. MoR's learned policy sits far below its own best uniform depth (3.515 vs 4.056); MoRE's sits just below its forced-depth-7 endpoint (3.637 vs 3.697). The dotted line is MoE, which has no recursion. This is the paper's central contrast: adaptive depth buys MoR a great deal and MoRE very little.

with the *worst* loss. The *alignment* of depth with difficulty is robust while the *amount* of compute skipped is not, and costs loss.

5.3 Where the composition does pay: robustness and efficiency

Out of domain the ordering inverts. The upper block of Table 1 scores each arm against the inference FLOPs it costs.

Three things follow.

In domain, all three arms are on the frontier; out of domain, MoRE dominates. Fig. 3 plots loss against inference compute. In domain no arm Pareto-dominates another: MoE is cheapest, MoR is best, MoRE sits between them, so MoRE's in-domain claim is a *ratio*—the most quality per unit of compute—and not a position the others fail to occupy. We are explicit about this because the weaker reading is the one a reviewer should be able to check and find already stated.

Out of domain the relationship changes in kind rather than degree. On held-out Pile blocks (Gao et al., 2021) MoRE reaches 7.573 nats against MoR's 7.802 and MoE's 8.272, at 2.19× MoE's FLOPs against MoR's 9.07×—so MoRE is both cheaper and better than MoR, and **MoR is Pareto-dominated**. Its advantage over MoE is 0.699 nats against MoR's 0.470, i.e. MoRE captures 149% of MoR's out-of-domain gain against 72% in domain. The in-domain ordering inverts, and this is the one respect in which composing the two mechanisms does something neither does alone.

It gets there far more cheaply, on a different axis. MoR pays 9.07× MoE's inference FLOPs; MoRE pays 2.19×—24% of MoR's total inference compute and 15% of its *extra* compute over MoE. Per FLOPs-multiplier MoRE buys 0.142 nats against MoR's 0.048: roughly 3× the quality per unit compute on validation loss, 6× out of domain. **We state the two axes separately and do not substitute one for the other**: at matched parameters MoR is lowest-loss by 0.1216 nats, which is significant at our resolution floor; per unit of inference compute MoRE is ahead. "Comparable to MoR" is therefore only ever a *per-FLOP* statement in this paper, never an absolute-quality one.

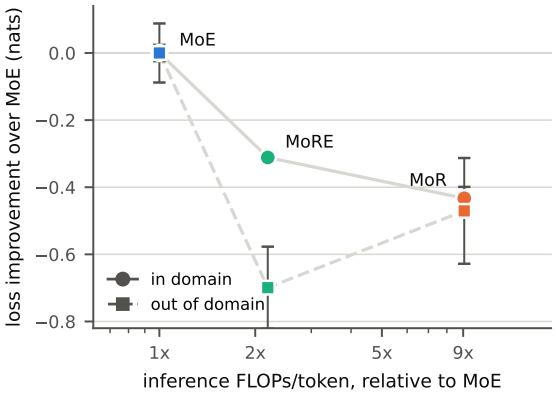


Figure 3: Loss *improvement over MoE* against inference FLOPs per token, five seeds, error bars one standard deviation; both domains share one scale because each is differenced against its own MoE baseline. In domain (solid) no arm dominates another—the three trace a frontier and MoRE's claim is the slope from MoE, not its position. Out of domain (dashed) MoR is *dominated*: MoRE improves more, at a quarter of the inference compute. Exploratory, uncorrected.

FLOPs, not wall-clock, is the compute measure. The throughput column keeps the latency cost vis-

Table 1: All results, five seeds, mean ± std. **Upper block:** every cell is *score (difference vs MoE, inference FLOPs× vs MoE, |difference| per FLOPs-multiplier)*; MoE is the reference arm, so its cells carry the score alone. The *score* is lower-better for loss rows and higher-better for accuracy; the third number is a **modulus**, so it is higher-better on *every* row—how much quality each unit of compute bought. Quality at matched parameters and quality per unit of compute are distinct axes and we never substitute one for the other. Validation loss is the primary metric (final-epoch; *best* equals *last* on all 15 cells, so no model selection occurred); pairwise, exact two-sided randomization test at the floor $p_{\min} = 0.00794$: MoRE−MoE -0.3113 ($d-15.9$), MoRE−MoR $+0.1216$ ($d+4.7$), MoE−MoR $+0.4329$ ($d+14.7$), all $p\,0.0079$. **Lower block:** model and cost facts. Δ floor is the margin below the bigram baseline of 4.9849 nats (positive *beats* it); FLOPs are inference FLOPs/token at each arm's measured mean depth (§4.3). The battery rows are **exploratory**—outside both declared families, uncorrected.

Eval	MoE	MoR	MoRE
Val loss (nats)	3.948 ± 0.024	**3.515** ± 0.034 (-0.433, 9.07×, 0.048)	3.637 ± 0.013 (-0.311, 2.19×, **0.142**)
Test loss (nats)	3.947 ± 0.022	**3.513** ± 0.033 (-0.435, 9.07×, 0.048)	3.621 ± 0.014 (-0.326, 2.19×, **0.149**)
OOD loss (nats)	8.272 ± 0.088	7.802 ± 0.158 (-0.470, 9.07×, 0.052)	**7.573** ± 0.122 (-0.699, 2.19×, **0.319**)
BLiMP (acc)	56.63 ± 1.37	**58.74** ± 1.39 ($+2.11$, 9.07×, 0.233)	58.36 ± 1.54 ($+1.73$, 2.19×, **0.791**)
Parameters	5,584,908	5,581,063	5,584,908
Val perplexity	51.9	**33.6**	38.0
Δ vs bigram floor	$+1.0366$	$+1.4695$	$+1.3480$
Inference FLOPs/token	**5.25 M**	47.6 M	11.5 M
Throughput (items/s)	**724**	170	170

ible: MoRE is no faster than MoR in wall-clock (170 items/s each) despite $4\times$ less arithmetic, because its per-token expert dispatch is latency-bound. A batched-dispatch implementation would be expected to realise the FLOP advantage; ours does not, and we claim no speedup.

Grammatical competence is real but modest, and routing does not buy it. On BLiMP (Warstadt et al., 2020) both recursive arms beat MoE (58.74% and 58.36% against 56.63%) and MoR is not distinguishable from MoRE ($p = 0.71$), so recursion buys measurable syntactic sensitivity and routing adds none. Appendix J reports it in full, with EWoK, on which every arm sits at chance. **Both recursive arms gain most where prediction is hardest, and least where context is shortest.** By target-token frequency decile the MoRE−MoE advantage grows from -0.147 nats on the most frequent to -0.374 on the rarest (MoR: -0.188 to -0.527), so neither gain is the uniform shift a pure capacity effect would give. By position, both arms gain most where context is *longest*—disconfirming the usual motivation for recursion (Appendix D).

6 Why the Composition Fails

Four accounts, labelled by evidential status: what is measured, and what is hypothesis consistent with the measurement.

6.1 Routing divides the data, not the update budget

[Ordering MEASURED; arithmetic MEASURED; mechanism HYPOTHESIS.] The obvious account of the ordering is capacity dilution: MoR's single block sees 100% of tokens while MoE and MoRE split the same FFN budget across six experts, so each expert is trained less. **That account is arithmetically false here, and ruling it out is what makes the surviving hypothesis worth stating.**

The budgets are matched by construction, and the matching cancels. Each MoE/MoRE expert FFN holds $2 \cdot 4 \cdot d^2 = 524{,}288$ weights and MoR's single FFN holds $2{\cdot}24{\cdot}d^2 = 3{,}145{,}728$, exactly six times as many (§3.4). A run sees 404.2M tokens. So:

per FFN parameter	MoE	MoR	MoRE
tokens seen	128.5	128.5	128.5
FFN applications	128	865	**894**

Tokens per parameter is *identical* across all three arms—the token split and the parameter split are both by E, so they cancel exactly. And once recursion is counted, applications per parameter slightly *favours* MoRE (894) over MoR (865), because MoRE runs marginally deeper. Whatever costs MoRE 0.12 nats against MoR, it is not that its parameters are trained less.

That comparison also explains the easy half of the ordering. MoRE gets $894/128 \approx 7\times$ the FFN applications per parameter that MoE gets, which is

simply what recursion buys, and MoRE beats MoE by 0.311 nats. The recursive multiply is real and it is the entire source of MoRE's advantage over MoE.

What remains, once update count is excluded, is the *diversity* of what each parameter block sees. MoR's block is exposed to the whole token distribution; each MoE/MoRE expert is exposed only to the router-selected slice that reaches it, which is both narrower and systematically biased, since the router selects it. Structure shared across the whole distribution—the bulk of what a language model must learn at this scale—can be learned once by MoR's block but has to be relearned independently inside all six experts. Six partial copies of shared structure fit in the same parameter budget as one complete copy only by making each copy worse.

This is a hypothesis, not an isolated effect, and we label it as one: separating it from the remaining differences needs a width sweep or a controlled manipulation of routing diversity, and we ran neither. It earns its place because it is the only account left after the arithmetic above excludes the update-count version, because it predicts the stratified result—MoRE's advantage over MoE grows with token difficulty (Appendix D.1), which is where an incompletely-learned shared structure should hurt most—and because it generates the falsifiable scale prediction in the Limitations section: as the corpus grows, each expert's slice eventually contains enough of the shared structure for the relearning penalty to amortise, so the MoR−MoRE gap should close with data scale at fixed parameters.

6.2 Depth saturation is a data-scale effect

[MEASURED, four points at fixed hyperparameters.] MoRE's halting sits at the ceiling on the full corpus (97.3% forced-exit, mean depth 6.955 of 7), inviting the reading that the architecture cannot learn adaptive depth. A subset sweep falsifies it: at 2.5%, 10% and 40% of the corpus the *same* model at the *same* ponder weight allocates depth adaptively (exit rates near zero, mean depth 1.9–2.4 of 7), and the collapse appears only at full scale. **The failure is a calibration finding, not an architectural one.** The axis is not cleanly monotone, so the honest phrasing is that saturation emerges between 40% and 100% of the data; the ponder term is linear in expected depth while the task gradient grows as the model fits, and once the halting sigmoid saturates its gradient vanishes (Graves, 2016). Banino et al. (2021) is the principled fix, unrun (Appendix H).

6.3 The two mechanisms compete for one budget

[MEASURED on a calibration grid; the canonical-corpus form is measured on the balance axis.] This is the paper's central claim, and it is the one that explains why the composition does not merely fail to add but subtracts.

Depth and specialisation are not independent knobs in MoRE the way they are in MoR and MoE separately. On the calibration grid, recovering adaptive depth by raising the halting weight destroys expert differentiation:

halting weight	mean depth	AMI	val loss
0.001 (frozen)	2.698	**0.2851**	5.2352
0.107	1.476	**0.0025**	5.3346
0.500	1.072	0.0207	5.3306

AMI of 0.0025 is statistically indistinguishable from the shuffled-label control. The mechanism is direct: fewer recursion steps means fewer expert applications per token, which means fewer opportunities for experts to differentiate. **Tune toward adaptive depth and specialisation collapses; preserve specialisation and depth saturates.** Both mechanisms draw on the same per-token compute budget, and at this parameter and data budget neither gets enough. (This grid is on a smaller dev corpus at one seed; it is a mechanism demonstration and carries that caveat.)

The competition has a second face: the balance term is load-bearing and cannot be traded for depth either—zeroing it collapses the load onto one expert and drives AMI to zero. Specialisation is real against per-run permutation controls (MoRE AMI 0.1803 vs control 0.0249, $z = 13.43$; MoE 0.1637 vs 0.0364) and the two arms are *not* distinguishable, so recursion does not improve routing just as routing does not preserve adaptive depth. Appendix K gives the controls.

We also tested, and falsified, the most attractive rival account. Subspace trapping—that routing once confines a token to one expert and repeated application drives the state to a fixed point—predicts lower dimensionality, oscillation and rescaling updates in the composed arm. All three fail: participation ratio is *higher* in the recur-

sive arms (57.3 and 70.3 against MoE 41.5), velocity decays monotonically, and updates are near-orthogonal (Appendix E). Excluding it leaves the budget competition as the account the data support.

7 Conclusion

At 5.6M parameters on WikiText-103, composing sparse expert routing with adaptive recursion buys efficiency and robustness rather than peak quality. All three arms lie on the quality/compute frontier; MoRE buys the most quality per unit of compute (72% of recursion's gain for 24% of its compute) and out of domain *dominates* recursion alone, reaching lower loss at a quarter of the compute—the one respect in which composing the mechanisms does something neither does alone. What it does not buy is in-domain quality: recursion alone leads by 0.1216 nats, significant at our resolution floor. The mechanism is not the obvious one—routing does not dilute the update budget, and subspace trapping fails three ways—but a competition for one per-token budget, in which the arm preserving specialisation saturates its depth. We release the engine, the evaluation battery, and the forced-depth control, and it is the last we would most like to see reused: it costs one inference pass, and it is what separates two arms that are indistinguishable by mean-and-histogram.

Limitations

Scale, stated as a falsifiable prediction

The study operates at 5.6M parameters, $d_{\text{model}} = 256$, one block, a vocabulary of 8,192, and three epochs of WikiText-103. Sparse-MoE benefits are known to emerge with scale, so a reviewer can reasonably say the negative result may be an artefact of operating below the regime where sparsity pays. **That objection is correct and cannot be answered with the runs that exist.**

Rather than hedge it, we convert it into the paper's forward claim. The account in §6.1 is that routing divides the *diversity* of data each parameter block sees, not its update budget—which the arithmetic there shows is matched exactly. That account predicts a specific crossover: as the corpus grows at fixed parameters, each expert's router-selected slice comes to contain enough of the distribution's shared structure that the re-learning penalty amortises, so **the MoR−MoRE gap should narrow monotonically with training data and the ordering should eventually invert in favour of the composition**. The out-of-domain result (§5.3) is weak early evidence in this direction, since MoRE is already the better arm on data it was not trained on.

The measurement that would falsify it: hold parameters, depth budget and optimiser fixed; sweep training tokens over at least a decade; and report the MoR−MoRE gap as a function of tokens per expert. If the gap is flat or widening, the diversity account is wrong and the composition's deficit is structural. We did not run that sweep, and a second model size—the single most valuable follow-up—is likewise unrun.

Budget, not convergence

Three epochs is a compute budget, and no arm is trained to convergence (§4.2). This interacts badly with exactly the mechanism the paper studies: §6.2 shows depth allocation is demonstrably budget-dependent, so the behaviour at 10 epochs is genuinely unknown and might differ in the direction that matters. Every arm receives the identical budget, so the comparison is internally valid; the generalisation to trained-to-convergence models is not established.

Relatedly, one hyperparameter configuration per arm was frozen from a grid run on a corpus roughly 60× smaller, and §6.2 demonstrates that at least one weight—the ponder penalty—did not transfer. We did not re-tune per arm, so the headline ordering could in principle be re-ordered by a per-architecture search. Given the gaps involved (0.12 nats between the recursive arms), we regard that as a real risk for the MoR–MoRE comparison and a small one for the comparisons against MoE.

Statistical resolution

$n = 5$ seeds puts every headline comparison exactly on the design's resolution floor of $p = 0.00794$ (§4.4). "As extreme as a five-seed design admits" is the strongest statement available, and it is weaker than significance. No larger effect could produce a smaller p here; only more seeds could. Cohen's d values are reported alongside every gap precisely because a large d on a small gap is the misreading to avoid—across our own re-exports, d has moved by a factor of 1.5 while the underlying gap moved by less than 0.005 nats.

What was declared and not run

One comparison in the pre-registered Family A, a depth-matched fixed-depth control, was declared before results existed and *still consumes its Holm slot*: we do not delete it to loosen the thresholds on the survivors. The forced-depth sweep in §5.2 answers the same question using inference passes over the canonical checkpoints rather than a trained control arm, so we report it as an exploratory mechanism result and leave the declared slot consumed and unfilled. A fixed-depth sweep trained from scratch, a shuffled-depth control, and a depth–input mutual-information analysis were all declared exploratory and are unrun.

Evaluation caveats

BLiMP and EWoK are scored with the tokenizer fitted to our training corpus (8,192 BPE merges), so **absolute accuracies are not comparable to published models** using 32k–50k vocabularies; only the cross-arm comparison is valid. EWoK sits within noise of its 50% chance floor for every arm, which we report as a scale limitation rather than a result—at 5.6M parameters a world knowledge probe is below its useful range. All evaluations outside the two declared families are uncorrected and carry no significance verdict in the abstract or conclusions.

The perplexities in Table 1 are intra-study quantities for the same reason and should not be compared to published WikiText-103 numbers.

Disclosures

Several facts about this study would count against it if a reader discovered them independently, so we state them.

An earlier version of the composed arm was not the architecture we define, and fixing it made our headline worse. It re-routed at every recursion step, letting a token traverse up to seven *different* expert FFNs—a heterogeneous seven-layer network, not one block iterated, and strictly more expressive than the specification allows. That arm scored 3.5031 and was statistically indistinguishable from MoR. The architecturally-correct route-once arm reported here scores 3.6370, **0.134 nats worse**. The earlier arm's apparent parity with MoR was an artefact of a specification violation. We retain it unmodified as a labelled ablation.

The pre-registration is partial and the registration file says so itself. MoE and MoR were already trained when the test families were declared, so only the comparisons involving the composed arm are genuinely pre-result. The MoR-versus-MoE comparison was specified after its data existed, and we do not describe the registration as complete.

The matrix spans two GPUs and four code revisions. Two arms were retrained on one card after the original checkpoints were lost; the composed arm remains on the original hardware, and comparability rests on a frozen specification and a seed-blind configuration-equality check rather than on a single revision. We have direct evidence the hardware split is benign: both retrained arms reproduced their published means to -0.117σ and -0.119σ—agreement to the third decimal across arms sharing no parameters, a *common-mode* offset rather than sampling noise, and one that cancels in the between-arm gaps carrying every verdict (the MoE$-$MoR gap moved by 0.000185 nats). Per-seed values do *not* reproduce and we do not claim they do; the reproducible quantity is the arm mean.

We report no per-token depth-allocation error. English supplies no ground-truth depth per token, so that metric is `N/A`—not zero. A zero would read as perfect allocation against a curriculum that does not exist.

Ethical Considerations

This work trains 5.6M-parameter language models on WikiText-103, a public corpus derived from Wikipedia, and evaluates them on public benchmarks. No human subjects, no annotation, and no personally identifying data are involved, and the models produced are far below the scale at which generative misuse is a practical concern. The realistic effect of the work runs through other researchers' decisions rather than through deployment: we report a negative result about an architecture combination, and a diagnostic—scoring any learned depth policy against a depth-matched uniform policy before describing it as adaptive—that should save compute otherwise spent on a mechanism that is not paying for itself. The symmetric risk is that a null obtained at 5.6M parameters discourages work that would succeed at scale; the Limitations section scopes the conclusion for exactly that reason and states the measurement that would overturn it. The corpus inherits the demographic and topical biases of Wikipedia, which we did not measure and which bear on any downstream

use of these checkpoints.

References

Sangmin Bae, Yujin Kim, Reza Bayat, Sungnyun Kim, Jiyoun Ha, Tal Schuster, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Aaron Courville, and Se-Young Yun. 2025. Mixture-of-recursions: Learning dynamic recursive depths for adaptive token-level computation. In *Advances in Neural Information Processing Systems (NeurIPS)*. ArXiv:2507.10524.

Andrea Banino, Jan Balaguer, and Charles Blundell. 2021. PonderNet: Learning to ponder. In *8th ICML Workshop on Automated Machine Learning (AutoML)*. ArXiv:2107.05407.

Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. 2019. Universal transformers. In *International Conference on Learning Representations (ICLR)*. ArXiv:1807.03819.

Maha Elbayad, Jiatao Gu, Edouard Grave, and Michael Auli. 2020. Depth-adaptive transformer. In *International Conference on Learning Representations (ICLR)*. ArXiv:1910.10073.

William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39.

Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. 2021. The Pile: An 825GB dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*.

Alex Graves. 2016. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*.

Anna A. Ivanova, Aalok Sathe, Benjamin Lipkin, Unnathi Kumar, Setayesh Radkani, Thomas H. Clark, Carina Kauf, Jennifer Hu, R. T. Pramod, Gabriel Grand, Vivian Paulun, Maria Ryskina, Ekin Akyürek, Ethan Wilcox, Nafisa Rashid, Leshem Choshen, Roger Levy, Evelina Fedorenko, Joshua Tenenbaum, and Jacob Andreas. 2024. Elements of world knowledge (EWoK): A cognition-inspired framework for evaluating basic world knowledge in language models. *arXiv preprint arXiv:2405.09605*.

Robert A. Jacobs, Michael I. Jordan, Steven J. Nowlan, and Geoffrey E. Hinton. 1991. Adaptive mixtures of local experts. *Neural Computation*, 3(1):79–87.

Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. 2021. GShard: Scaling giant models with conditional computation and automatic sharding. In *International Conference on Learning Representations (ICLR)*. ArXiv:2006.16668.

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2017. Pointer sentinel mixture models. In *International Conference on Learning Representations (ICLR)*. ArXiv:1609.07843.

David Raposo, Sam Ritter, Blake Richards, Timothy Lillicrap, Peter Conway Humphreys, and Adam Santoro. 2024. Mixture-of-depths: Dynamically allocating compute in transformer-based language models. *arXiv preprint arXiv:2404.02258*.

Tal Schuster, Adam Fisch, Jai Gupta, Mostafa Dehghani, Dara Bahri, Vinh Q. Tran, Yi Tay, and Donald Metzler. 2022. Confident adaptive language modeling. In *Advances in Neural Information Processing Systems (NeurIPS)*. ArXiv:2207.07061.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*. ArXiv:1701.06538.

Nguyen Xuan Vinh, Julien Epps, and James Bailey. 2010. Information theoretic measures for clusterings comparison: Variants, properties, normalization and correction for chance. *Journal of Machine Learning Research*, 11(95):2837–2854.

Alex Warstadt, Alicia Parrish, Haokun Liu, Anhad Mohananey, Wei Peng, Sheng-Fu Wang, and Samuel R. Bowman. 2020. BLiMP: The benchmark of linguistic minimal pairs for English. *Transactions of the Association for Computational Linguistics*, 8:377–392.

Barret Zoph, Irwan Bello, Sameer Kumar, Nan Du, Yanping Huang, Jeff Dean, Noam Shazeer, and William Fedus. 2022. ST-MoE: Designing stable and transferable sparse expert models. *arXiv preprint arXiv:2202.08906*.

A The resolution floor of the randomization test

The argument is worth stating because the floor is a property of the design rather than of the data. Our statistic is $|\mathrm{mean}(a) - \mathrm{mean}(b)|$. When the two arms are the same size, the complement of any admissible group-a subset is itself an admissible group-a subset, and swapping the groups negates the difference while preserving its absolute value. Every regrouping at the observed extreme therefore has a partner that is equally extreme, the count can never be odd at the tail, and the smallest attainable fraction is $2/\binom{n_a+n_b}{n_a}$ rather than $1/\binom{n_a+n_b}{n_a}$. When the arms differ in size the complement has the wrong size, is not an admissible regrouping, no mirror is forced, and $1/\binom{n_a+n_b}{n_a}$ becomes attainable.

B Multiple-comparison correction

Correction is Holm–Bonferroni over two *pre-registered* families, declared before the final arm was trained: Family A (predictive quality, $m = 4$, metric validation task loss) and Family B (adaptive depth, $m = 2$, metric depth–difficulty correlation). The split is declared rather than chosen because the two families ask different questions on different quantities, and pooling them would correct the depth claim for tests that cannot bear on it. Holm controls the same familywise error rate as plain Bonferroni and is uniformly more powerful, and here the choice is load-bearing: plain Bonferroni over the seven comparisons would give $0.05/7 = 0.00714$, *below* the floor of 0.00794—a design in which no effect of any size could be declared significant. Holm's easiest threshold at $m = 4$ is 0.0125, a margin of 0.0046 above the floor.

Two consequences we hold ourselves to. A declared comparison that could not be run still consumes its slot: dropping a failed arm must never make the survivors easier to call significant. And a comparison whose floor exceeds its Holm threshold is *undecidable* and reported `N/A`, never "not significant"—that phrasing would blame the data for a limit of the design.

C Integrity checks and structurally-zero loss terms

C.1 Integrity checks

Three mechanical invariants hold across the matrix, each enforced by an automated check rather than by inspection. **(i) Inputs are bit-identical across arms for the same record**: feature ranges are identical at $E \in \{1, 5, 6\}$, so a difference between arms cannot be a difference in representation. **(ii) The regression or classification target is absent from the input**, and no oracle routing label is encoded into a feature slot. **(iii) The arms are parameter-matched to 0.069%** and the seed-blind configuration is identical within each arm, so comparability cannot rest on a single code revision: the matrix spans four commits, and it is the frozen specification plus the config-equality check that holds the arms comparable, not one revision.

One disclosure belongs with the config table. Three loss terms are structurally zero on language: the family-classification term because its head reads a detached representation so its gradient cannot reach the trunk, and the routing-supervision and halting-supervision terms because natural language supplies no per-token oracle expert and no per-token oracle depth. The first is a genuine advantage over a comparable synthetic study, where a family cross-entropy is often one of the larger terms in the objective and every specialisation claim must be caveated for it.

D Loss stratified by token frequency and position

D.1 Stratified loss: the advantage concentrates on hard tokens

Fig. 4 groups validation loss by the frequency decile of the *target* token—difficulty being a property of what must be predicted, not of the input—and by position within the block.

Both recursive arms improve most where prediction is hardest. The MoRE−MoE advantage grows from -0.147 nats on the most frequent decile to -0.374 on the rarest; MoR's from -0.188 to -0.527. The widening is not monotone—the gap dips at the seventh and tenth deciles—so we claim a trend, not a monotone relationship. Neither architecture's gain is a uniform shift, which is what a pure capacity effect would look like.

A prediction this does not support. If recursion were doing what the adaptive-computation premise suggests—spending steps where context is thin—the recursive arms should gain most at the *start* of a block. They do the opposite: MoRE's advantage over MoE is -0.047 nats in the first 8 positions and -0.359 in the last 128; MoR's goes from -0.250 to -0.465. Both recursive arms gain most where context is *longest*. We report this because it disconfirms a mechanism we would otherwise have asserted, and because it is consistent with the capacity account in §6: a shared block iterated is accumulating representation, not compensating for missing context.

E The subspace-trapping probes

E.1 A plausible mechanism, refuted

[MEASURED; hypothesis falsified.] The natural explanation for MoRE's collapse is *subspace trapping*: routing a token once confines its recursion to a single expert's parameter subspace, and repeated application of one block within that subspace drives the state toward a fixed point rather than refining it. It is an appealing account and it is wrong. We tested it three ways.

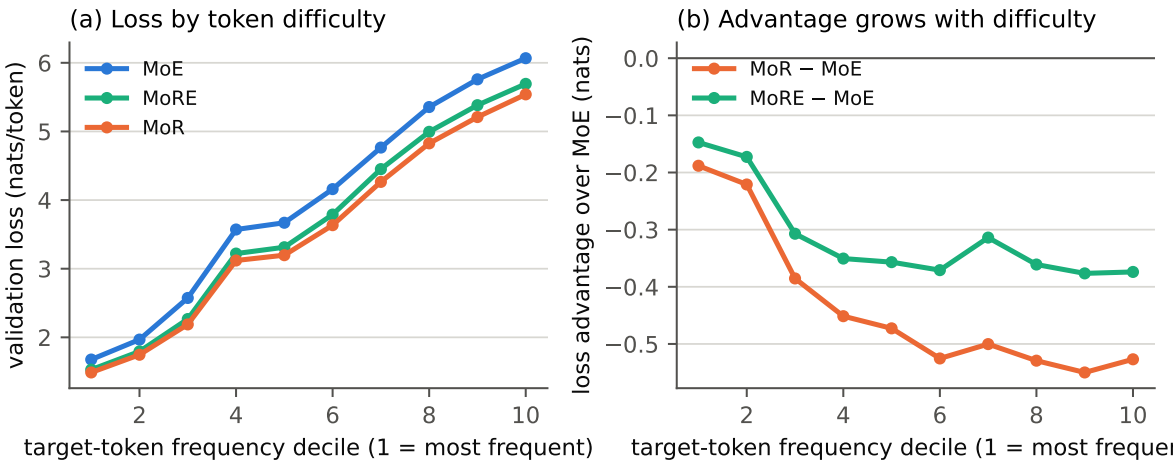


Figure 4: (a) Validation loss by target-token frequency decile; decile 1 is the *most frequent* (easiest) and decile 10 the rarest. (b) Each recursive arm's advantage over MoE, which widens toward the rarer deciles, though not monotonically. Exploratory, uncorrected.

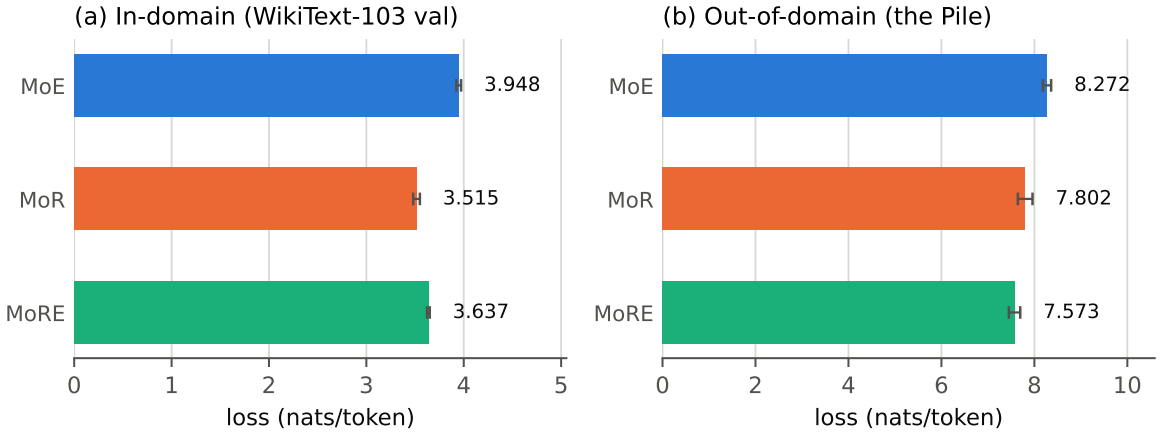


Figure 5: The ordering inverts out of domain: MoRE is worst in domain (a) and best on the Pile (b). Same three entities and colours in both panels. Exploratory, uncorrected.

probe	MoE	MoR	MoRE
participation ratio	41.5	**70.3**	57.3
inter-expert cosine	0.630	N/A	0.916

Participation ratio is the final-state value, ± 1.6, 5.6 and 2.8 across seeds respectively; the cosine is the mean between-expert value.

(i) Trapping predicts lower participation ratio in the composed arm; it is higher. If recursion were collapsing the state onto a low-dimensional subspace, the composed model's final-state PR should fall below MoE's, since MoE has no recursion at all. It does not: MoRE's PR is 57.3 against MoE's 41.5, and MoR's—with no routing—is higher still at 70.3. Recursion *expands* the effective dimensionality, and routing contracts it somewhat without collapsing it. The ordering refutes the prediction.

(ii) Trapping predicts oscillating or fixed-point dynamics; velocity decays smoothly. Measuring the L2 norm of consecutive post-norm states across depth, both recursive arms decay monotonically—MoR from 11.06 at step 2 to 5.71 at step 7, MoRE from 9.09 to 3.49—which is the signature of settling, not of vibrating around an attractor.

(iii) Trapping predicts that successive updates rescale the state; they are near-orthogonal. The mean cosine between a step's update and the state it updates from is negative in both arms (-0.26 to -0.06), i.e. updates push into new directions rather than amplifying existing ones. A rescaling collapse would show positive collinearity approaching $+1$.

The recursion in this architecture is dynamically healthy: it travels, it settles, and it adds dimensions.

The failure is not a broken iterated map. That rules out the most attractive explanation and leaves the budget competition (§6.3) as the account the data supports—which is a stronger position than an unexplained null, because we can say which of the obvious mechanisms it is *not*.

F Matching the parameter budget across the three arms

F.1 One engine, three modes

MoE, MoR and MoRE are configurations of one implementation with an explicit `architecture` mode, not three codebases. They share one data pipeline, one optimiser setup and one evaluation path, so a difference between them cannot be a difference in the harness. Only the fields their definitions require differ: $E \in \{6, 1, 6\}$, $T \in \{1, 7, 7\}$, MoR's FFN multiplier (below), and the two loss weights for quantities that do not exist in a given arm (no ponder cost at depth 1, nothing to balance with one expert).

We write **MoR** for our recursion-only configuration—one shared block with adaptive halting—which is in the spirit of weight-tied recursive transformers with learned halting (Dehghani et al., 2019; Bae et al., 2025) but is not a reproduction of any specific published system; all three arms are configurations of the same engine.

Matching the budget. MoR has one FFN where MoRE has six, so matching the per-FFN multiplier would compare 5.6M parameters against 0.9M and call the difference architectural. We instead equalise *total* FFN hidden width, giving MoR's single FFN a multiplier of $24 = 4 \times 6$. The result is 5,584,908 parameters for MoE and MoRE, identical, and 5,581,063 for MoR—a 0.069% deficit that is six per-expert halt heads, the router and FFN biases. That residual is irreducible at any integer multiplier, and it runs *against* the headline result: the arm with slightly fewer parameters is the one that wins in domain.

G Metric definitions

G.1 Metrics

Primary metric: validation task loss, the final-epoch per-token cross-entropy in nats. We deliberately do not select on best-epoch validation loss: that is an order statistic whose downward bias scales with each arm's per-epoch validation noise, which is not matched across architectures. On all 15 cells `best` equals `last`—the evaluation interval is one epoch and the curves decrease monotonically—so *no model selection occurred* on either split. Best-epoch values are reported alongside so the selection effect stays checkable.

Because the router is unsupervised, expert indices carry no semantic identity, so raw agreement with any externally defined partition measures label permutation as much as partition quality. We therefore report AMI (Vinh et al., 2010), Hungarian-matched accuracy and cluster purity, each against a *per-run permutation control*: the same metric computed against shuffled linguistic labels, averaged over 10 draws. The controlled quantity is the z-score of the real value against that null, not the raw value. The externally defined partition is a part-of-speech hypothesis about a useful expert split—a hypothesis, not a functional ground truth—so a router can be better for next-token prediction and still score low against it. We therefore never call this "routing accuracy" without that qualifier.

For depth we report mean steps, exit rates, halt mass, and the Spearman correlation between per-token depth and per-token difficulty. **The correlation is only interpretable when depth has variance to correlate**: on an arm whose tokens almost all exit at the ceiling, the correlation is computed against a near-constant and a value near zero is uninformative rather than negative evidence. We report the exit rates next to every correlation for that reason (§5.2).

H The subset sweep behind the depth-saturation finding

H.1 Depth saturation is a data-scale effect, not an architectural limit

[MEASURED, four points at fixed hyperparameters.] MoRE's halting sits at the ceiling on the full corpus (97.3% forced-exit, mean depth 6.955 of 7). It is tempting to read that as the architecture being unable to learn adaptive depth. A subset sweep falsifies that reading.

training data	mean depth	forced exit
2.5%	2.43	≈ 0
10%	1.92	≈ 0
40%	1.97	≈ 0
100% (canonical)	6.96	0.973

At every reduced-data point the same model, at the same ponder weight, allocates depth adaptively—exit rates near zero, mean depth between 1.9 and 2.4, inside a budget of 7. The col-

lapse appears only at full scale. **So the failure is a calibration finding, not an architectural one**: the ponder penalty that produces adaptive depth on a small corpus cannot hold depth down at this one.

We state the limit of that claim precisely, because the sweep is exploratory (single seed, subset fraction below 1, and it may never be quoted as a canonical result). The subset axis is *not* cleanly monotone—2.5% gives 2.43 but 40% gives 1.97—so the honest phrasing is that *saturation emerges somewhere between 40% and 100% of the data*, not that depth is monotone in training steps. The mechanism is consistent: the ponder term is linear in expected depth, while the task gradient grows as the model fits, so the fixed penalty is outpaced. Once the halting sigmoid saturates, its gradient vanishes and the linear-penalty ACT scheme cannot recover on its own (Graves, 2016). Banino et al. (2021) replaces the linear penalty with a KL divergence against a geometric prior, which keeps a gradient on the halt head near saturation, and is the principled fix; we did not run it, and it is named as future work rather than as a result.

I Training protocol and hyperparameters

I.1 Training protocol

The frozen dataset identifier is `lang-wikitext-103-bpe8192-len256-800d6154`, and the split is the corpus's own author-provided validation split.

Every canonical arm runs at seeds $\{42, 43, 44, 45, 46\}$ and is reported as mean $\pm$ std over five runs. Training is 3 epochs of AdamW (lr 10^{-3}, weight decay 10^{-4}, gradient clipping 1.0, dropout 0.1) with $d_{\text{model}} = 256$, one block, 4 attention heads, $T = 7$ and batch size 48 on a single laptop GPU. These settings were fixed once, before the comparison, and not swept per architecture: they are a stable operating point, and no arm was tuned for itself.

Three epochs is a fixed compute budget, not convergence, and we state it as such. One epoch is 526,320 blocks $\times$ 256 tokens $=$ 134.7M tokens against $\approx$3.4M non-embedding parameters, so the runs sit in an underfitting regime; the choice of 3 over 2 was made because the halting axis needs more than two epochs to separate the arms at all, and going to 8 epochs was rejected on cost. Every arm receives the identical budget, so the comparison is valid even though no arm is converged. The consequences are stated in §7.

J BLiMP and EWoK, in full

Grammatical competence is real but modest, and routing does not buy it. On BLiMP (Warstadt et al., 2020), 67,000 minimal pairs, both recursive arms beat MoE (58.74% and 58.36% against 56.63%) and MoR is not distinguishable from MoRE ($p = 0.71$). Every arm is above the 50% floor, so recursion buys measurable syntactic sensitivity at this scale; adding routing does not add more of it. The gains concentrate on long-distance agreement phenomena—anaphor agreement and determiner–noun agreement—which is where a shared iterative block would be expected to help. We also ran EWoK (Ivanova et al., 2024), on which all three arms sit within noise of the 50% chance floor (49.62, 50.33, 49.95); that is a statement about the scale of these models, not about the architectures, so we exclude it from Table 1 and draw no comparison from it.

K Specialisation against per-run controls

The competition has a second face on the canonical corpus, along a different axis. The routing balance term is load-bearing and cannot be traded away for depth either: setting it to zero *collapses* the expert load onto a single expert (load entropy 0.0001) and drives AMI to zero. A near-uniform per-token router distribution coexisting with balanced load and above-chance specialisation is therefore not a pathology to be tuned out—and we note that we initially read it as one before the sweep refuted us (§7).

Specialisation itself is real, and measured against per-run controls rather than against chance. MoRE's AMI is 0.1803 ± 0.0246 against a permutation control of 0.0249, a controlled z of 13.43; Hungarian-matched accuracy is 0.3592 ± 0.0271 against a control of 0.2464. MoE's AMI is 0.1637 ± 0.0478 against a control of 0.0364 ($z = 6.82$), Hungarian 0.3616 against 0.2966. Both six-expert arms find substantial above-chance structure—and, read together, the two arms are not distinguishable from each other. Routing quality is approximately the same with recursion as without it, which is the routing half of the interference result: recursion does not improve routing, and routing does not preserve adaptive depth, in the same architecture.